

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2 int main()
3 {
4     int m,n,i,j;
5     printf("Enter number of memory blocks:\n");
6     scanf("%d",&m);
7     int block[m], original[m];
8     printf("Enter size of each block:\n");
9     for(i=0;i<m;i++)
10    {
11        scanf("%d",&block[i]);
12        original[i] = block[i];
13    }
14    printf("Enter number of processes:\n");
15    scanf("%d",&n);
16    int process[n];
17    printf("Enter size of each process:\n");
18    for(i=0;i<n;i++)
19        scanf("%d",&process[i]);
20    for(i=0;i<n;i++)
21    {
22        int bestIndex = -1;
23        for(j=0;j<m;j++)
24            {

```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

Select Language : C (GCC 9.2.0)

```

23     for(j=0;j<m;j++)
24     {
25         if(block[j] >= process[i])
26         {
27             if(bestIndex == -1 || block[j] < block[bestIndex])
28             {
29                 bestIndex = j;
30             }
31         }
32     }
33     if(bestIndex != -1)
34     {
35         int fragment = block[bestIndex] - process[i];
36         printf("Process %d of size %d is allocated to block %d\n", i+1, process[i], bestIndex+1, originalBlock[bestIndex]);
37         block[bestIndex] = fragment;
38     }
39     else
40     {
41         printf("Process %d of size %d is not allocated\n", i+1, process[i]);
42     }
43 }
44 return 0;
45 }
46 
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success ✓

Input :

```

5
100 500 200 300 600
4
212 417 112 426

```

Expected Output :

```

Enter number of
memory blocks:
Enter size of each
block:
Enter number of
processes:
Enter size of each
process:

```